
Tecnologie del Linguaggio Naturale

Riassunto – Parte 2 (Radicioni)

Università degli Studi di Torino (UniTO)
Corso di Laurea Magistrale in Informatica
A.A. 2025/26

Alessandro Olivero

Argomenti: KR, WordNet, WSD, N-grammi, VSM, Word2Vec, Transformers

Documento generato a scopo di studio

0. Indice

1. Introduzione alla Semantica Lessicale e al NLP

La **semantica lessicale** studia il significato delle parole, le relazioni tra di esse, e come rappresentare e computare questi significati.

Le quattro generazioni dei modelli linguistici

Era	Tipo	Approccio	Metodi chiave
anni '90	Statistical LM	Assunzione di Markov	N-grammi, smoothing
2013	Neural LM	Reti neurali	Word2Vec (rappresentazioni distribuite)
2018	Pre-trained LM	Transformer	BERT, GPT (bidirezionale/unidirezionale)
2020+	Large LM	Scaling + prompting	GPT-3/4, ChatGPT, Claude

Livelli di analisi linguistica

1. **Fonetica/fonologia**: suoni e loro combinazioni.
2. **Morfologia**: struttura interna delle parole (morfemi, flessione, derivazione).
3. **Sintassi**: struttura delle frasi (costituenti, dipendenze).
4. **Semantica**: significato delle parole e delle frasi.
5. **Pragmatica**: uso del linguaggio nel contesto (speech acts, implicature).
6. **Discorso**: struttura testuale, coerenza, coreference.

Ambiguità nel linguaggio naturale

- **Morfologica/PoS**: “back” = ADJ / NOUN / VERB / ADV.
- **Sintattica**: PP attachment, coordinazione (vedi Parte 1).
- **Semantica**: polisemia (“banca”), scope quantificazionale.
- **Fonetica**: omofoni.

Esempio classico (Mazzei/Radicioni): “I made her duck.”

- *duck* = sostantivo (anatra) oppure verbo (abbassarsi)
- *make* = causativo (le ho fatto abbassare) oppure creativo (le ho cucinato l’anatra)

2. Knowledge Representation (KR) e Ontologie

Ontologia: definizione

Il termine *ontologia* viene dal greco: *on* (essere) + *logos* (discorso). In filosofia studia *cosa esiste* e la struttura della realtà. In informatica:

Ontologia (informatica): specificazione esplicita e formale di una *concettualizzazione*.

Concettualizzazione: struttura formale di una porzione della realtà così co-

me è percepita e organizzata da un agente, *indipendentemente* dal vocabolario usato e dall'occorrenza di situazioni specifiche.

Livelli di precisione ontologica

In ordine crescente di formalizzazione:

1. **Catalogo**: lista di termini (minima precisione).
2. **Glossario**: lista con definizioni informali.
3. **Thesaurus**: glossario + relazioni lessicali (sinonimia, antonimia).
4. **Tassonomia**: classificazione gerarchica (relazioni *is-a*).
5. **Schema DB / OO**: struttura formale con proprietà e vincoli.
6. **Teoria assiomatica**: specificazione logica completa (massima precisione).

Esempio – dominio Tennis: catalogo $\rightarrow$ {tennis, campo, rete} $\rightarrow$ tassonomia
gioco_atletico $\rightarrow$ gioco_di_campo $\rightarrow$ tennis $\rightarrow$ teoria: $tennis(x) \Rightarrow gioco_campo(x) \wedge \exists y[campo(y) \wedge giocato_in(x, y)]$.

Componenti di un'ontologia

- **Componente terminologica**: classi, relazioni, assiomi – indipendente da situazioni specifiche.
- **Componente assertiva**: fatti individuali – riflette stati epistemici specifici.

DOLCE – Descriptive Ontology for Linguistic and Cognitive Engineering

DOLCE è un'ontologia di alto livello (upper ontology) che fornisce un framework per descrivere entità del mondo reale.

Distinzione fondamentale: Endurant vs Perdurant

- **Endurant (Oggetto)**: è *completamente presente* in qualsiasi istante in cui esiste. Può cambiare nel tempo mantenendo la propria identità. Es.: una rosa.
- **Perdurant (Evento)**: non è completamente presente in ogni istante; si *svolge* nel tempo e non cambia (tutte le sue parti sono essenziali). Es.: una conferenza, una performance musicale.

Relazione: gli Endurant *partecipano* nei Perdurant.

Identità e criteri di identità

Un criterio di identità determina cosa rende unica un'entità. Esempio: **time-duration** è identificata dalla lunghezza; **time-interval** è identificata da inizio e fine. Pur avendo parti comuni, *non* sono in relazione is-a perché hanno criteri di identità diversi.

Qualità, Quale e Quality Space

- **Quality (particolare)**: il colore di *questa specifica* rosa.
- **Quale (valore)**: una specifica tonalità di rosso.
- **Quality Space (universale)**: l'insieme di tutti i valori di colore possibili.
- Relazione: la qualità *inheres in* (dipende da) l'entità.

Tassonomia DOLCE (semplificata)

Categoria	Sottocategorie
ENDURANT	Fisico (oggetti, materia) / Non-fisico (mentale, sociale)
PERDURANT	Statico (State, Process) / Dinamico (Achievement, Accomplishment)
QUALITY	Fisica (spaziale, ...) / Temporale
REGION	Quality region, Space region, Time region

La relazione part-of

La relazione di *parthood* ha diverse varianti:

- Componente (il manico è componente della porta)
- Ingrediente (la farina è ingrediente del pane)
- Porzione (una fetta di pane)
- Area (una città in una regione)
- Membro (una nave in una flotta)
- Pezzo (un manico staccato)

3. WordNet: Struttura e Applicazioni

Cos'è WordNet

WordNet (Miller et al., 1990–) è un **database lessicale** dell'inglese che organizza nomi, verbi, aggettivi e avverbi per *significato* anziché per forma.

Dimensioni: ~117.000 synset, ~155.000 coppie parola-senso.

Il concetto di Synset

Un **synset** (synonym set) è un insieme di parole che esprimono lo stesso concetto in un dato contesto. Ogni parola può appartenere a più synset (polisemia).

Esempio: `car.n.01` = {car, auto, automobile, machine, motorcar}; ognuno esprime lo stesso concetto di “veicolo a motore a quattro ruote”.

Relazioni lessicali e semantiche

Relazione	Tipo	Esempio
Sinonimia	Lessicale	car = automobile
Antonimia	Lessicale	good $\neq$ bad
Iponimia	Semantica	car is-a vehicle
Iperonimia	Semantica	vehicle is-hypernym-of car
Meronimia	Semantica	wheel is-part-of car
Olonimia	Semantica	car has-part wheel
Entailment	Semantica	to buy entails to pay
Causazione	Semantica	to kill causes to die
Similar-to	(agg.)	happy <i>similar-to</i> joyful

Misure di similarità su WordNet

Misure path-based

Shortest Path Similarity:

$$\text{sim}_{\text{path}}(w_1, w_2) = \frac{1}{\text{shortestPath}(w_1, w_2) + 1}$$

Limite: non considera la profondità della gerarchia (due nodi vicini in cima alla gerarchia sembrano molto simili).

Leacock-Chodorow (LC):

$$\text{sim}_{\text{LC}}(w_1, w_2) = -\log\left(\frac{\text{shortestPath}}{2 \times D}\right)$$

dove D è la profondità massima della tassonomia.

Wu-Palmer (WUP) – usata nel corso:

$$\text{sim}_{\text{WUP}}(w_1, w_2) = \frac{2 \times \text{depth}(\text{LCA})}{\text{depth}(w_1) + \text{depth}(w_2)}$$

dove LCA = Lowest Common Ancestor (antenato comune più basso). Range: $[0, 1]$. Premia le coppie con LCA profondo.

Esempio Wu-Palmer:

Coppia	depth(w_1)	depth(w_2)	depth(LCA)	sim _{WUP}
PUG – POODLE	3	3	2 (DOG)	4/6 = 0.667
PUG – SIAMESE	3	3	1 (ANIMAL)	2/6 = 0.333

Misure basate sull'Information Content (IC)

L'**Information Content** di un concetto c è:

$$\text{IC}(c) = -\log P(c)$$

dove $P(c)$ è la probabilità che una parola nel corpus sia iponimo di c .

Resnik:

$$\text{sim}_{\text{Res}}(w_1, w_2) = -\log P(\text{LCA}) = \text{IC}(\text{LCA})$$

Considera solo il LCA; non dipende dalla distanza tra w_1 , w_2 e il LCA.

Lin:

$$\text{sim}_{\text{Lin}}(w_1, w_2) = \frac{2 \times \text{IC}(\text{LCA})}{\text{IC}(w_1) + \text{IC}(w_2)}$$

Versione normalizzata di Resnik. Range: $[0, 1]$.

Jiang-Conrath:

$$\text{sim}_{\text{JC}}(w_1, w_2) = \frac{1}{\text{IC}(w_1) + \text{IC}(w_2) - 2 \times \text{IC}(\text{LCA})}$$

4. Word Sense Disambiguation (WSD)

Definizione del problema

La **polisemia** è la proprietà per cui una parola ha più significati. Il WSD è il task di determinare *quale* significato una parola ha in un dato contesto.

Esempio: “bank” nel testo “I deposited money in the bank” → istituto finanziario (non riva del fiume).

Baseline: assegnare sempre il senso più frequente (Most Frequent Sense, MFS) → ~65% di accuratezza su inglese.

Algoritmo di Lesk (1986)

L'algoritmo di Lesk è un metodo **knowledge-based** che sfrutta le definizioni del dizionario.

Idea: il senso corretto è quello la cui definizione (gloss) presenta la maggior *sovrapposizione* con il contesto della parola da disambiguare.

Algoritmo:

1. Per ogni senso s della parola w :
2. Ottieni il gloss $G(s)$ dal dizionario (WordNet).
3. Calcola l'overlap con il contesto C della frase: $\text{score}(s) = |G(s) \cap C|$.
4. Ritorna il senso con score massimo.

$$\hat{s} = \arg \max_s |\text{gloss}(s) \cap \text{context}(w)|$$

Esempio: “bank” in “I deposited money in the bank”:

- Gloss senso 1 (finanziario): “financial institution for money and deposits”
- Gloss senso 2 (fiume): “edge of a river”
- Contesto: {deposited, money, bank}
- Overlap: senso 1 » senso 2 → scelto senso 1.

Varianti:

- **Extended Lesk:** include definizioni delle parole correlate (iponimi, iperonimi).
- **Normalized Lesk:** divisione per la lunghezza del gloss.

Metodi graph-based

Costruzione di un grafo dei sensi candidati:

- Nodi = sensi candidati di tutte le parole nella frase.
- Archi = relazioni semantiche in WordNet (con pesi = similarità).
- Propagazione del contesto tramite algoritmi come **PageRank**.

$$\text{rank}(s) = \frac{1-d}{N} + d \sum_{t \in \text{vicini}(s)} \frac{\text{rank}(t)}{\text{outdegree}(t)}$$

dove $d \approx 0.85$ è il damping factor.

Metodi knowledge-based (combinazione di sensi)

Per ogni frase, si cercano le assegnazioni di sensi $\{s_1, \dots, s_k\}$ che massimizzano la similarità complessiva tra tutte le coppie:

$$\hat{S} = \arg \max_{\{s_1, \dots, s_k\}} \sum_{i \neq j} \text{sim}(s_i, s_j)$$

Metodi supervisionati

Usano corpus annotati con i sensi corretti (SemCor, SemEval). Feature: parole vicine, PoS tag, collocazioni. Classificatori: SVM, Naive Bayes, reti neurali, BERT fine-tuned. Problema: bottleneck dell’annotazione (pochi dati per sensi rari).

Sfide del WSD

- Sparsità dei dati (rari sensi poco osservati).
- Distinguere polisemia (stessa parola, sensi correlati) da omografia (stessa forma, origini diverse: “bank” finanziario vs geografico).
- Distinzioni fine-grained difficili (es. significati molto simili).

- Corpus di addestramento limitati per lingue non inglesi.

5. Text Processing: Pipeline di Base

Tokenizzazione

La **tokenizzazione** suddivide il testo in unità minime significative (token).

- **Word tokenization:** separazione per spazi e punteggiatura.
- **Sentence tokenization:** separazione in frasi (‘.’, ‘!’, ‘?’).
- **Subword tokenization:** BPE, WordPiece (per Transformers).

Sfide: contrazioni (*don’t*), trattini (*mother-in-law*), numeri (*\$1,234.56*), abbreviazioni (*Dr.*), emoji, social media (hashtag, mention).

Implementazione NLTK:

```
from nltk.tokenize import word_tokenize, sent_tokenize
tokens = word_tokenize("Hello, world!") # ['Hello', ',', ' ', 'world', '!', '']
frasi = sent_tokenize("Prima. Seconda.") # ['Prima.', 'Seconda.']
```

Normalizzazione

- **Lowercasing:** “The” → “the”. Riduce la sparsità; perde info su propri.
- **Rimozione punteggiatura/caratteri speciali:** dipende dal task.

Stemming

Lo **stemming** riduce le parole alla radice (stem) applicando regole morfologiche.

Porter Stemmer (algoritmo più comune): applica regole in cascata per rimuovere suffissi.

Input	Output (stem)
inflations	inflat
consolingly	consol
computing	comput

Limiti: over-stemming (“computer” e “computation” → “comput”); under-stemming; output non è una parola valida.

Lemmatizzazione

La **lemmatizzazione** riduce una parola al suo *lemma* (forma canonica del dizionario), sfruttando informazioni morfologiche e il PoS tag.

Input	PoS	Lemma
running	VERB	run
better	ADJ	good
better	VERB	better
was	VERB	be

```
from nltk.stem import WordNetLemmatizer
lem = WordNetLemmatizer()
lem.lemmatize("running", pos="v") # "run"
lem.lemmatize("better", pos="a") # "good"
```

Stemming vs Lemmatizzazione

Aspetto	Stemming	Lemmatizzazione
Input richiesto	Parola	Parola + PoS
Output	Stem (può non essere valido)	Lemma (parola valida)
Velocità	Veloce	Lento
Precisione	~70%	~95%
Uso tipico	Information Retrieval	NLP avanzato, analisi

Rimozione Stopword

Le **stopword** sono parole molto frequenti con poco contenuto semantico (“a”, “the”, “is”, “and”, ...). La loro rimozione riduce il rumore e accelera il processing; tuttavia può eliminare informazioni importanti per certi task (es. “not”).

```
from nltk.corpus import stopwords
stop = set(stopwords.words('english'))
tokens_filtrati = [t for t in tokens if t not in stop]
```

6. N-grammi e Modelli di Linguaggio

Definizioni

Un **n-gramma** è una sequenza di n token consecutivi.

Unigramma ($n = 1$): singola parola.

Bigramma ($n = 2$): coppia di parole consecutive.

Trigramma ($n = 3$): tripla di parole consecutive.

Assunzione di Markov

Chain rule esatta:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdots P(w_n|w_1, \dots, w_{n-1})$$

Approssimazione di Markov (per bigrammi):

$$P(w_1, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$$

Ogni parola dipende solo dalla precedente (*finestra di memoria limitata*).

Stima MLE (Maximum Likelihood Estimation)

$$P_{\text{MLE}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Problema fondamentale: se la coppia (w_{i-1}, w_i) non è mai apparsa nel corpus di training, $P = 0$ e la probabilità dell'intera frase collassa a zero.

Laplace Smoothing (Add-1)

$$P_{\text{Lap}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

dove $|V|$ è la dimensione del vocabolario. Nessuna probabilità zero, ma ridistribuisce eccessivamente la massa di probabilità dagli eventi frequenti a quelli non visti.

Interpolazione lineare

Combina modelli di ordine diverso per maggiore robustezza:

$$P_{\text{interp}}(w_i | w_{i-1}) = \lambda_1 P(w_i) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i | w_{i-2}, w_{i-1})$$

con $\lambda_1 + \lambda_2 + \lambda_3 = 1$. I λ si ottimizzano su un corpus di development.

Kneser-Ney Smoothing

Il metodo più efficace in pratica. Introduce il concetto di **frequenza di continuazione**: quante parole diverse seguono w_{i-1} ? Questo penalizza le parole che, pur frequenti in assoluto, appaiono sempre nello stesso contesto.

$$P_{\text{KN}}(w_i | w_{i-1}) \propto \max(C(w_{i-1}, w_i) - d, 0) + \lambda(w_{i-1}) \cdot P_{\text{KN}}(w_i)$$

dove $d \in (0, 1)$ è un parametro di discounting e $P_{\text{KN}}(w_i)$ è la probabilità di *continuazione* (proporzione di bigrammi distinti in cui w_i appare come secondo elemento).

Perplexity

La **perplexity** misura quanto un modello è “sorpreso” da un testo di test: più bassa è, meglio il modello rappresenta i dati.

$$PP(W) = P(w_1 w_2 \cdots w_n)^{-1/n} = \left(\prod_{i=1}^n \frac{1}{P(w_i | w_{i-1})} \right)^{1/n}$$

Equivalentemente, tramite cross-entropy:

$$PP(W) = 2^{H(W)} \quad \text{dove} \quad H(W) = -\frac{1}{n} \sum_{i=1}^n \log_2 P(w_i | w_{i-1})$$

Interpretazione: media geometrica dei fattori di inversione di probabilità. Un modello perfetto darebbe $PP=1$. Valori tipici: 30–100 per buoni modelli su testo inglese.

Log-Likelihood

$$LL(W) = \sum_{i=1}^n \log P(w_i | w_{i-1})$$

Sempre ≤ 0 . Più alto (meno negativo) = modello migliore. Più stabile della perplexity per la classificazione (evita divisioni per zero e overflow).

Type-Token Ratio (TTR)

$$TTR = \frac{|\text{Types}|}{|\text{Tokens}|}$$

Misura la ricchezza lessicale. Alto TTR = vocabolario vario (slang, abbreviazioni come nei Twitter). Basso TTR = testo ripetitivo (stile narrativo formale).

7. Lab 4 – N-grammi: Twitter vs Testo Letterario

Obiettivo

Confrontare le proprietà statistiche e generative di due corpora molto diversi: tweet (social media) e prosa letteraria (Emma di Jane Austen, 1816).

Dataset e preprocessing

- **Twitter:** 10.000 tweet (5.000 positivi + 5.000 negativi) da `nlk.corpus.twitter_samples`.
- **Letterario:** `austen-emma.txt` da `nlk.corpus.gutenberg`.

Pulizia Twitter: rimozione URL (`http\S+`), mention (`@\w+`), hashtag (`#`), caratteri non alfabetici.

Pulizia letterario: rimozione punteggiatura, lowercase.

Pipeline principale (codice)

```

# Modello MLE (n=2) per generazione testo
train, vocab = padded_everygram_pipeline(2, [tokens_tweet])
modello = MLE(2)
modello.fit(train, vocab)

# Modello Laplace per perplexity (gestisce OOV)
train_lp, vocab_lp = padded_everygram_pipeline(2,
    [tokens_tweet])
modello_lp = Laplace(2)
modello_lp.fit(train_lp, vocab_lp)

# Probabilita' condizionale
p = modello.score('love', ['i']) # P(love | i)

# Generazione testo
testo = modello.generate(30, random_seed=42)

# Perplexity (normalizzata per vocabolario)
pp = modello_lp.perplexity(word_tokenize(frase))
pp_norm = pp / len(modello_lp.vocab)

```

Classificatore con Log-Likelihood

```

def log_likelihood(modello, tokens):
    score = 0.0
    for i in range(1, len(tokens)):
        prob = modello.score(tokens[i], [tokens[i-1]])
        score += math.log(prob) if prob > 0 else
            math.log(1e-10)
    return score

# Regola di decisione: vince il modello con LL piu' alta
ll_tw = log_likelihood(modello_tweet, tokens)
ll_lett = log_likelihood(modello_lett, tokens)
verdetto = "TWITTER" if ll_tw > ll_lett else "LETTERARIO"

```

Risultati e conclusioni

Metrica	Twitter	Letterario
Token totali	~120.000	~160.000
Parole uniche (types)	~20.000	~8.000
TTR	~0.17 (alto)	~0.05 (basso)
Lunghezza media frase	~10 parole	~22 parole

Osservazioni chiave:

- Twitter ha TTR più alto: vocabolario vario (slang, abbreviazioni, neologismi).
- Le frasi letterarie sono più lunghe e strutturate sintatticamente.

- MLE con OOV dà perplexity = ∞ : il Laplace smoothing è *indispensabile*.
- Il classificatore **log-likelihood** è più stabile e accurato della perplexity normalizzata.
- Per $n = 3$ il modello soffre di sparsità (trigrammi non visti): $n = 2$ è il miglior trade-off.

8. Vector Space Model (VSM)

Bag-of-Words (BoW)

Ogni documento diventa un vettore di frequenze di parole, ignorando l'ordine:

$$d = [c(\text{word}_1, d), c(\text{word}_2, d), \dots, c(\text{word}_V, d)]$$

Limite: ignora l'ordine, peso uniforme a tutte le parole.

TF-IDF

TF-IDF bilancia la frequenza locale (TF) con la rarità globale (IDF).

$$\text{TF}(t, d) = \frac{C(t, d)}{|d|} \quad \text{IDF}(t) = \log \frac{N}{\text{DF}(t)} \quad \text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

dove N = numero totale di documenti, $\text{DF}(t)$ = numero di documenti che contengono t .

Effetti: termini comuni in molti documenti (“the”, “is”) $\rightarrow$ IDF basso $\rightarrow$ TF-IDF basso. Termini rari e discriminativi $\rightarrow$ IDF alto $\rightarrow$ TF-IDF alto.

Cosine Similarity

$$\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \cdot \|d_2\|} = \frac{\sum_i d_1[i] \cdot d_2[i]}{\sqrt{\sum_i d_1[i]^2} \cdot \sqrt{\sum_i d_2[i]^2}}$$

Range: $[0, 1]$ per vettori non negativi. 0 = vettori ortogonali (completamente diversi); 1 = stessa direzione. La *normalizzazione* rende la misura invariante alla lunghezza del documento.

Rocchio's Algorithm (Relevance Feedback)

Aggiorna il vettore di query avvicinandolo ai documenti rilevanti e allontanandolo da quelli non rilevanti:

$$Q' = \alpha \cdot Q + \beta \cdot \frac{1}{|R|} \sum_{D \in R} D - \gamma \cdot \frac{1}{|NR|} \sum_{D \in NR} D$$

Valori tipici: $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.25$.

Latent Dirichlet Allocation (LDA)

LDA è un modello probabilistico generativo per l'**analisi tematica** dei documenti.

Ipotesi: ogni documento è una miscela di topic; ogni topic è una distribuzione su parole.

Processo generativo:

1. Per ogni documento d : estrai la distribuzione sui topic $\theta_d \sim \text{Dirichlet}(\alpha)$.
2. Per ogni posizione di parola nel documento:
 - a. Estrai il topic $z \sim \text{Multinomial}(\theta_d)$.
 - b. Estrai la parola $w \sim \text{Multinomial}(\beta_z)$.

Vantaggi: interpretabile, non supervisionato, gestisce polisemia (una parola è assegnata a topic diversi in contesti diversi). **Limiti:** bag-of-words (ignora l'ordine), numero di topic fissato a priori, lento.

9. Word2Vec: Rappresentazioni Distribuite

Ipotesi distribuzionale

Firth (1957): “You shall know a word by the company it keeps.”
Parole che appaiono in contesti simili hanno significati correlati.

Architetture Word2Vec

CBOW (Continuous Bag of Words)

Input: le parole di contesto (finestra di $\pm w$ parole). **Output:** la parola target centrale.

1. One-hot encode le parole di contesto.
2. Embedding layer: lookup + media degli embedding.
3. Output layer: softmax sul vocabolario.
4. Loss: cross-entropy con la parola target.

Vantaggio: veloce, efficiente su grandi corpora.

Skip-gram

Input: la parola target. **Output:** le parole di contesto. Inverso del CBOW.

1. One-hot encode la parola target.
2. Embedding layer: lookup embedding target.
3. Per ogni parola di contesto: output layer softmax.
4. Loss: somma delle cross-entropy per ogni parola di contesto.

Vantaggio: funziona meglio per parole rare; migliore qualità degli embedding su corpora piccoli.

Negative Sampling

Problema: il softmax sull'intero vocabolario ($O(|V|)$) è troppo costoso.

Soluzione: per ogni campione positivo ($w_{\text{target}}, w_{\text{contesto}}$), campionare k parole *negative* casuali e ottimizzare:

$$\mathcal{L} = \log \sigma(u_t \cdot v_c) + \sum_{i=1}^k \log \sigma(-u_t \cdot v_{n_i})$$

dove σ è la funzione sigmoide. Riduce la complessità da $O(|V|)$ a $O(k)$. Tipicamente $k = 5-20$.

Proprietà degli embedding: word arithmetic

$$\begin{aligned}\vec{v}(\text{king}) - \vec{v}(\text{man}) + \vec{v}(\text{woman}) &\approx \vec{v}(\text{queen}) \\ \vec{v}(\text{Paris}) - \vec{v}(\text{France}) + \vec{v}(\text{Germany}) &\approx \vec{v}(\text{Berlin})\end{aligned}$$

Dimostra che lo spazio degli embedding codifica relazioni semantiche e sintattiche come trasformazioni lineari.

Parametri di training

Parametro	Descrizione	Valore tipico	Codice
vector_size	Dimensione embedding	100–300	vector_size=100
window	Finestra di contesto ($\pm w$)	5–10	window=5
min_count	Frequenza minima	2–5	min_count=3
epochs	Passi sul corpus	5–20	epochs=20
sg	0=CBOW, 1=Skip-gram	–	sg=1
negative	Negative samples	5–20	negative=5

Implementazione Gensim

```
from gensim.models import Word2Vec

model = Word2Vec(
    sentences=testi_tokenizzati, # lista di liste di token
    vector_size=100, window=5, min_count=3,
    epochs=20, sg=1, seed=42
)

# Recupero embedding
vec = model.wv['love'] # array di 100 float

# Similarita' coseno tra due parole
```



```
sim = model.wv.similarity('love', 'heart') # float in [-1,1]

# Parole piu' simili
simili = model.wv.most_similar('king', topn=5)
# [('queen', 0.87), ('prince', 0.82), ...]
```

Analisi della deriva semantica (Semantic Shift)

Metrica di overlap tra i k vicini più simili in due ere diverse:

$$\text{overlap}(w, \text{era}_1, \text{era}_2) = \frac{|\text{vicini}_{\text{era}_1}(w) \cap \text{vicini}_{\text{era}_2}(w)|}{k}$$

Interpretazione: $\text{overlap} \approx 1$ = significato stabile; $\text{overlap} \approx 0$ = deriva semantica completa.

Delta similarità tra coppie di parole:

$$\Delta(w_1, w_2) = |\text{sim}_{\text{era}_1}(w_1, w_2) - \text{sim}_{\text{era}_2}(w_1, w_2)|$$

Limiti di Word2Vec

- **Embedding statici:** ogni parola ha un unico vettore, indipendentemente dal contesto (non gestisce la polisemia).
- **Finestra limitata:** considera solo il contesto locale ($\pm w$ parole).
- **Nessuna struttura sub-word:** “walking” e “walk” sono appresi indipendentemente.
- **Bias:** apprende pregiudizi presenti nel corpus di addestramento.

10. Lab 8 – Word2Vec su Testi Musicali (70s vs 2000s)

Obiettivo

Analizzare il cambiamento semantico del vocabolario musicale tra gli anni '70 e gli anni 2000 tramite due modelli Word2Vec separati.

Dataset e artisti

Era	Artisti
Anni '70	Led Zeppelin, Pink Floyd, Queen, Eagles, Fleetwood Mac, ...
Anni 2000	Eminem, Coldplay, Linkin Park, Taylor Swift, Lady Gaga, ...

Dataset: `spotify_millsongdata.csv`.

Pipeline di preprocessing

```
def pulisci_testo(testo):
    testo = re.sub(r'\[.*?\]', '', testo)    # rimuove
        [Chorus], [Verse]
    testo = re.sub(r'^a-zA-Z\s]', '', testo) # solo
        lettere
    testo = testo.lower()
    tokens = testo.split()
    # rimuove stopwords e token corti (< 3 caratteri)
    return [t for t in tokens if t not in stop and len(t)
        >= 3]
```

Addestramento e analisi

```
params = dict(vector_size=100, window=5, min_count=3,
               workers=4, epochs=20, seed=42)
model_70s = Word2Vec(df_70s['tokens'].tolist(),
                    **params)
model_2000s = Word2Vec(df_2000s['tokens'].tolist(),
                      **params)

# Parole simili per era
for parola in ['love', 'night', 'heart']:
    sim70 = [w for w, _ in
              model_70s.wv.most_similar(parola, topn=3)]
    sim00 = [w for w, _ in
              model_2000s.wv.most_similar(parola, topn=3)]
    print(f"{parola}: 70s={sim70} 2000s={sim00}")

# Analisi deriva: overlap dei vicini
for parola in parole_comuni:
    vicini_70 = set(w for w, _ in
                    model_70s.wv.most_similar(parola, topn=10))
    vicini_00 = set(w for w, _ in
                    model_2000s.wv.most_similar(parola, topn=10))
    overlap = len(vicini_70 & vicini_00) / 10
    cambiamento = 1 - overlap
```

Clustering K-Means + t-SNE

```
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE

# Prendi top-200 parole per frequenza
parole = model_70s.wv.index_to_key[:200]
vettori = np.array([model_70s.wv[w] for w in parole])

# K-Means con k=6 cluster
km = KMeans(n_clusters=6, random_state=42, n_init=10)
labels = km.fit_predict(vettori)
```

```
# t-SNE per visualizzazione 2D (100D -> 2D)
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
coords_2d = tsne.fit_transform(vettori)
```

Risultati e conclusioni

- **“love”**: negli anni '70 contesto romantico/metaforico; negli anni 2000 più diretto e personale.
- **“night”**: negli anni '70 mistero e oscurità; negli anni 2000 vita notturna urbana.
- Le parole caratteristiche degli anni '70 riflettono il rock psichedelico; quelle degli anni 2000 il rap e la musica elettronica.
- I cluster K-Means mostrano temi coerenti con i generi musicali delle rispettive ere.
- La **heatmap di similarità** evidenzia come le relazioni tra parole chiave (love, heart, god, soul...) cambino tra le due ere.
- Metrica di overlap: una parola con overlap < 50% ha subito una deriva semantica significativa.

11. Transformers e Rappresentazioni Contestuali

Limiti di Word2Vec che motivano i Transformers

Word2Vec assegna a ogni parola *un unico vettore*, indipendentemente dal contesto. Non gestisce la polisemia:

- “I went to the **bank** to withdraw cash.” (banca finanziaria)
- “I sat on the **bank** of the river.” (riva del fiume)

Entrambe le occorrenze di “bank” ricevono lo stesso vettore in Word2Vec.

Meccanismo di Attention

L'**attention** permette di pesare diversamente le parti dell'input in base al loro ruolo nel calcolo del significato di una specifica parola target.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Dove:

- Q (Query): cosa stiamo cercando?
- K (Key): cosa offre ogni posizione?
- V (Value): l'informazione da recuperare.

- $\sqrt{d_k}$: fattore di scala (evita gradienti saturi nel softmax).

Intuitivamente: per la parola “bank” in una frase finanziaria, il softmax darà pesi alti a parole come “withdraw”, “deposit”, “money”; in un contesto fluviale darà pesi alti a “river”, “water”, “sit”.

Multi-Head Attention

Esegue h meccanismi di attention in parallelo con proiezioni apprese diverse:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Vantaggio: ogni “testa” impara relazioni diverse (sintattiche, semantiche, coreference, ...).

BERT (Bidirectional Encoder Representations from Transformers)

BERT è un Transformer **bidirezionale**: vede tutto il contesto (sia a sinistra che a destra) durante il pre-training.

Pre-training task:

1. **Masked Language Modeling (MLM)**: il 15% dei token è mascherato; il modello deve predire il token originale.
 - 80% sostituito con [MASK]
 - 10% sostituito con parola casuale
 - 10% lasciato invariato
2. **Next Sentence Prediction (NSP)**: dato [CLS] A [SEP] B [SEP], predire se B segue A nel testo.

Uso in downstream tasks:

1. Aggiungere token speciali: [CLS], [SEP].
2. Passare attraverso BERT: embedding contestualizzati per ogni token.
3. Il vettore [CLS] è usato come rappresentazione dell'intera frase.
4. Fine-tuning sul task specifico.

GPT (Generative Pre-trained Transformer)

GPT è un Transformer **unidirezionale** (causale): può vedere solo i token precedenti, non quelli successivi.

Pre-training: language modeling autoregressivo:

$$\mathcal{L} = - \sum_t \log P(w_t \mid w_1, \dots, w_{t-1})$$

BERT vs GPT: confronto

Aspetto	BERT	GPT
Direzione	Bidirezionale	Unidirezionale (sinistra→destra)
Pre-training	MLM + NSP	Language Modeling autoregressivo
Punti di forza	Comprensione, classificazione	Generazione testo
Embedding	Contestuali bidirezionali	Contestuali causali

Embedding contestuali vs statici

Con BERT, ogni occorrenza di una parola riceve un vettore *diverso* in base al contesto circostante:

```
# Le due occorrenze di "bank" avranno embedding diversi
text1 = "I went to the bank to withdraw cash"
text2 = "I sat on the bank of the river"
# embedding1(bank) != embedding2(bank) <- gestisce
# polisemia!
```

12. Riepilogo: Confronti tra Approcci

Stemming vs Lemmatizzazione

	Stemming	Lemmatizzazione
Tool	Regole morfologiche	Morfologia + dizionario
Input	Parola	Parola + PoS
Output	Radice (può non essere valida)	Lemma (parola valida)
Precisione	~70%	~95%
Velocità	Molto veloce	Lento

MLE vs Laplace vs Kneser-Ney

	MLE	Laplace	Kneser-Ney
OOV	$P = 0$ (fallisce)	$P > 0$	$P > 0$
Over-smoothing	No	Sì (eccessivo)	No
Complessità	Bassa	Bassa	Alta
Uso	Generazione	Perplexity, classif.	LM di produzione

WordNet vs Word2Vec vs BERT

Aspetto	WordNet	Word2Vec	BERT
Costruzione	Manuale	Automatica (corpus)	Automatica (corpus)
Polisemia	Esplicita (synset)	Non gestita	Gestita (contestuale)
Similarità	Graph-based	Coseno	Coseno contestuale
Relazioni	Esplicite (is-a, ...)	Implicite	Implicite
Copertura	Limitata (~155k)	Dipende dal corpus	Vasta
Aggiornamento	Curato	Data-driven	Data-driven

N-grammi vs Word2Vec vs Transformers

Aspetto	N-grammi	Word2Vec	Transformer
Contesto	Limitato (n parole)	Finestra fissa	Intera sequenza
Polisemia	No	No (statico)	Sì (contestuale)
Training	Rapido	Moderato	Lento
Parametri	Pochi	$ V \times d$	Miliardi
Interpretabilità	Alta	Bassa	Molto bassa
Downstream NLP	Limitato	Buono	Eccellente

13. Formule di Riferimento

Formula	Nome
$P_{\text{MLE}}(w ctx) = C(ctx, w)/C(ctx)$	MLE bigrammi
$P_{\text{Lap}}(w ctx) = (C(ctx, w) + 1)/(C(ctx) + V)$	Laplace smoothing
$PP(W) = (\prod_i 1/P(w_i w_{i-1}))^{1/n}$	Perplexity
$LL(W) = \sum_i \log P(w_i w_{i-1})$	Log-Likelihood
$TTR = \text{Types} / \text{Tokens} $	Type-Token Ratio
$TF(t, d) = C(t, d)/ d $	Term Frequency
$IDF(t) = \log(N/DF(t))$	Inv. Document Freq.
$\cos(d_1, d_2) = (d_1 \cdot d_2)/(\ d_1\ \ d_2\)$	Cosine Similarity
$\text{sim}_{\text{WUP}}(w_1, w_2) = 2 \text{dep}(\text{LCA})/(\text{dep}(w_1) + \text{dep}(w_2))$	Wu-Palmer
$\text{IC}(c) = -\log P(c)$	Information Content
$\text{sim}_{\text{Lin}}(w_1, w_2) = 2 \text{IC}(\text{LCA})/(\text{IC}(w_1) + \text{IC}(w_2))$	Lin
$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k})V$	Scaled Dot-Product
$\mathcal{L}_{\text{neg}} = \log \sigma(u_t \cdot v_c) + \sum_i \log \sigma(-u_t \cdot v_{n_i})$	Negative Sampling